

Kagecal Progress Report

Decentralized Calendar

Samuel Neisewander Leonardo Molina
sneisewa@nd.edu lmolina3@nd.edu

Repository: <https://github.com/samneisewanderND/kagecal>

1 Purpose

The Kage Bunshin no Calendar (Shadow Clone Calendar) is a decentralized calendar application where several peers can manipulate a single shared calendar timeline. Upon connection, the peer synchronizes with the most up-to-date calendar in the system. Once synchronized, the peer is able to Create, Read, Update, or Delete (CRUD) events on the shared calendar timeline. Any change made by one peer is propagated to all peers to maintain a consistent view of the shared calendar. Once a peer exits, it loses access to the current shared calendar until it rejoins the system. The system is intended for anyone who wants to run a shared calendar locally among a small group of peers without relying on a centralized cloud service.

1.1 Goals

The project has 3 main goals, which we discuss in order of importance:

1. **Consistency:** All peers operate on their own copy of the calendar, so the system naturally suffers from potential inconsistency. A communication protocol is required to serialize peer operations and maintain a synchronized calendar. Thus, the main goal of the project is to create a system that is consistent when a sequence of CRUD operations is performed on the shared calendar. Specifically, a peer should never lose a calendar event operation performed by itself or by others. Each operation should be atomic; it either completes fully or has no effect. The system should be able to recover from peer crashes without corrupting the calendar state.
2. **Resilience and Fault Tolerance:** The system must gracefully handle peers leaving the system, whether voluntarily or due to crashes. If a leader peer fails, the remaining peers must be able to elect a new leader and resume normal operations. When a crashed peer comes back online, it must be able to recover its state from its local transaction log and checkpoint, and then synchronize with the current leader to obtain the latest calendar state.
3. **Scalability and Performance:**
 - (a) **Throughput:** The system should be able to handle a reasonable number of concurrent CRUD operations without excessive delay, even as the number of peers increases.
 - (b) **Latency:** Individual CRUD operations should complete in a reasonable amount of time. The voting protocol introduces communication overhead for each operation performed on the calendar; thus, minimizing round-trip times between peers and the leader is important.

1.1.1 Challenges

The project faces 3 main challenges:

1. **Balancing Consistency and Performance:** The main goal of consistency is also the main challenge of the project. Constructing a communication protocol that maintains the logical ordering of events among all peers in the system is difficult on its own. This protocol becomes more challenging when scalability and performance are added to the formula. These goals are at odds with each other, because consistency requires coordination (writing to disk, waiting for peer votes), while performance demands that each operation is completed quickly.
2. **Leader Election:** The election algorithm is a challenge that stems directly from the challenge of consistency. Ensuring that a single leader is always available to coordinate operations, even when peers join, leave, or crash, requires a detailed protocol design. The election algorithm must cover a wide range of edge cases, such as concurrent elections and network partitions (split-brain).
3. **State Restoration:** When a peer crashes and later rejoins, or when a new leader is elected, the system must restore diverging calendar states. This is done by determining the peer with the most up-to-date state and propagating that state to other peers; this is a nontrivial synchronization problem.

1.2 Distinction from existing work

This project is similar to distributed calendars provided by large tech companies such as Google Calendar and Outlook Calendar.

- **Shared Calendar Module:** In Kagecal, there is one calendar shared among all peers, rather than one peer having an individual calendar with sharing of specific events.
- **Feature set:** The calendar's feature set is limited in scope to fit the semester timeline. Some notable features that are missing are location lookups for meetings (Google Maps, Apple Maps), alerts and reminders, and support for multiple calendars per user.
- **Architecture:** Unlike commercial calendar services that rely on large-scale distributed databases and data centers, Kagecal uses a single elected leader peer to handle all CRUD operations. This simplifies the design but limits the scalability compared to production systems.

2 Architecture

2.1 CRUD operations

There are 4 operations that are being performed on the calendar, which are Create, Read, Update, Delete (CRUD), each described as follow:

1. **Create:** Create a new event on a peer's local calendar. Each event has 3 required arguments (**name**, **start**, and **end**) and 3 optional arguments (**description**, **location**, and **repeats**). For most of the arguments, they are self-explanatory. The repeats argument takes in **Repeats** class with three fields: **repeats_every** (a set of days from an enumerated type), **repeats_starting** (a UTC timestamp for the first occurrence), **repeats_until** (a UTC timestamp for the last occurrence).

2. **Read:** Reads an event from the shared calendar. This operation has different variants: `get_event()` retrieves a single event by identifier, and `list_events()` returns all events.
3. **Update:** Modifies an existing event on the shared calendar via event identifier. Each modification takes the same 6 arguments from `create` plus a required event identifier to specify which event to modify.
4. **Delete:** Deletes an event from the shared calendar by event identifier.

2.2 System Design

The entire system is split into 5 files: *Calendar.py*, *PersistentCalendar.py*, *Client.py*, *Server.py*, *Peer.py*, and *kagecal.py*. The following subsections describe each module and how they work together to build the decentralized calendar.

2.2.1 Calendar: *Calendar.py*

The calendar exposes methods that perform operations on the local calendar. Internally, events are stored in an in-memory hash table mapping event identifiers to event data. The methods are: `create`, `delete`, `modify`, `get_event`, and `list_events`.

2.2.2 Persistence: *PersistentCalendar.py*

This module extends the Calendar module to provide persistence via a checkpoint, log, and update file. The maximum number of transactions before compaction is 100 entries. When the compaction threshold is reached, a new checkpoint containing the calendar state and logical clock is written to a temporary file on disk. The new checkpoint is atomically renamed, overwriting the previous checkpoint (if it exists). The transaction log and transaction counter are cleared. Any operation that mutates the calendar state (`create`, `delete`, `modify`) is appended to the transaction log; incrementing the logical clock per transaction is implied when replaying the log.

The log is appended to for every write operation in the calendar (`Create`, `Delete`, and `Modify`). The log appends transactions that contain three fields: `method`, `identifier`, and `event`. The method indicates which operation the transaction represents. The event and identifier are an event class instance and its identifier, respectively. This approach provides durability; if a peer crashes, it can restore from its checkpoint and replay the transaction log to recover its calendar state before the crash. After recovery, the peer synchronizes with the current leader to obtain any changes that occurred while it was offline.

The update file is used when the calendar state and logical clock need to be atomically written to disk on a peer sync. The process involves calling down to the persistent calendar module and passing down the new logical clock and calendar state. From there, the module creates the update file atomically before calling checkpoint. After checkpointing, the peer is synced with the most up-to-date calendar, and the update file is removed. Now, if the peer crashes and comes back online, it will restore from the most up-to-date calendar state in the network.

2.2.3 Client: *Client.py*

The client is responsible for placing outgoing RPCs to a single peer. This module contains RPC stubs for every method in the *Calendar* API, as well as some RPCs used for elections and synchronization. These include *who_is_leader*, *call_election*, *coordinate*, and *sync*, which get the endpoint of the current leader, call an election, coordinate followers with the new leader, and synchronize with

a peer's calendar state, respectively. The client places RPCs via a TCP connection with its target peer, serializing messages in JSON format before sending. If the client cannot reach a specific peer, it will resend the request using exponential backoff until it reaches a configurable number of retry attempts. After the maximum number of retries, the layer above the client assumes the peer is offline, which may result in an election if the target peer was the leader.

2.2.4 Server: *Server.py*

The server operates in one of two modes: leader or follower mode. As a leader, the server will listen for CRUD operation requests from followers, serialize them, and commit them to its own calendar by calling down to the PersistentCalendar module. To totally order operations, the leader will maintain a logical clock for each of its socket connections, and both the leader and its peers will include this clock in RPC messages. This way, if an RPC is retried and arrives at the server twice, the second arrival will be ignored, assuming the first was committed and acknowledged. For each CRUD operation performed on the calendar, the leader performs a broadcast of the new logical clock. This broadcast is an indication for every peer in the system to query the leader for the most up-to-date calendar state. In follower mode, the server will not commit incoming RPCs from other peers to its own calendar; instead, it will only answer requests having to do with the state of the system (i.e., election messages or leader location messages). This functionality is exposed to callers of the *Server* module by the method *serve*. In the background, the server registers itself with the ND catalog server (a centralized name server) every minute via UDP. The leader election protocol is described in more detail in Section 2.3.

2.2.5 Putting it all together *Peer.py*

The peer module combines the server with several client instances to provide the full functionality of the distributed calendar. Client applications can call down to this peer module to read and write the shared calendar state as if it were a simple local calendar. When a peer starts, it first attempts to locate other peers by contacting endpoints found on the catalog nameserver. A fresh client instance is created for each peer connection. If a listening peer is found, the module asks who the current leader is, sets the server to follower mode, and performs RPCs normally. If no listening peers are found, the server initiates an election, which will result in it running in leader mode. The peer calls down to the server module to interact with the local copy of the calendar, and calls down to the client module to operate on the shared calendar. In the event of a crash, the persistence layer ensures that the peer can recover to its last recorded state, at which point it can contact the network and ask the leader for changes to the shared calendar state. The entire system is shown in figure 1.

2.2.6 Name Server

The system utilizes the ND catalog server as a centralized name server for peer registration and discovery. Each peer periodically updates the name server every minute using UDP, sending its type, owner, port, project name, peer identifier, and PID. The project is the same across all peers participating in the same shared calendar. This design allows for multiple independent shared calendars to coexist and improves testing. Each peer identifier is a universally unique identifier (UUID) to guarantee uniqueness.

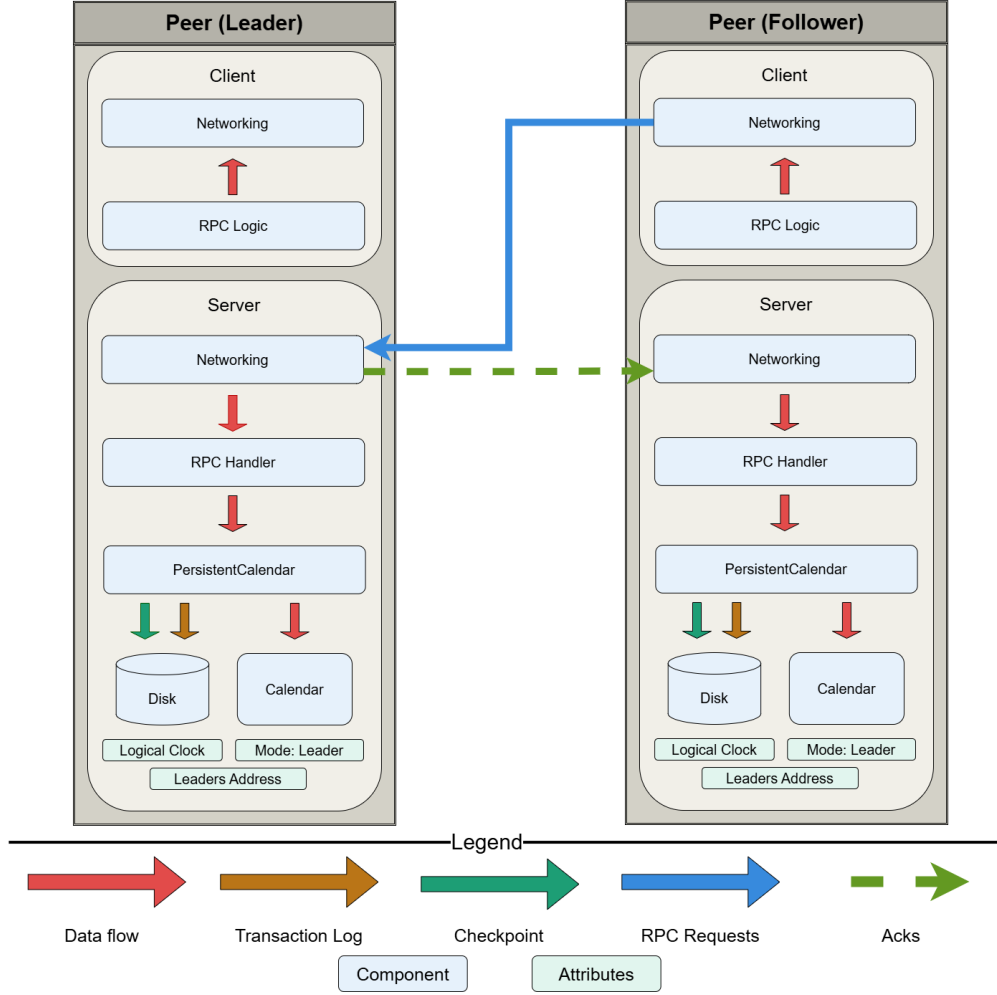


Figure 1: Network IO and Internal Module in Peer

2.2.7 Application: kagecal

The kagecal module is the user interface application that connects the users to the shared calendars. Constructing the system into several modules allowed the user application the ability to multiplex several calendar states. This is managed by the UI thread, which takes in user input. The user has the option to `join` and `switch` between different shared calendars. Additionally, the application exposes CRUD options to perform operations on the shared calendars.

2.3 Leaders & Election Algorithm

The system uses the bully algorithm for leader election. In the system, each peer has a unique process identifier (PID). The peer with the highest PID becomes the leader among active peers. Since this application can be run on separate machines, a tiebreaker was broken by the peers' addresses. This algorithm was chosen for its simplicity and its effectiveness at electing a single leader to coordinate all write operations. A detailed walk-through of the election algorithm is shown in Figure 2.

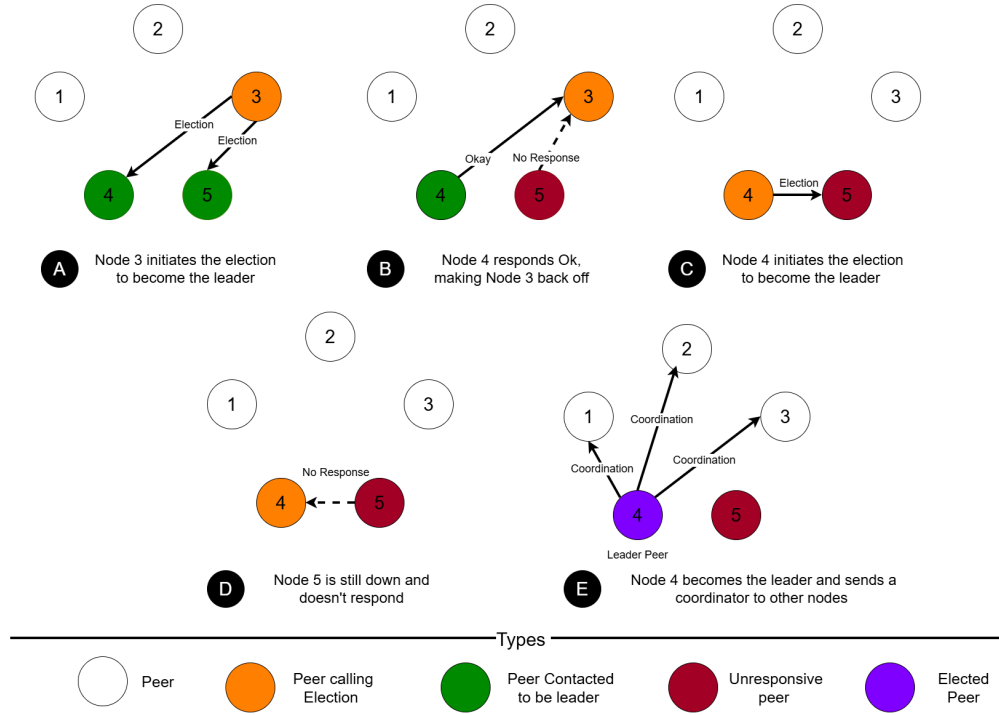


Figure 2: Peer Election Algorithm

2.3.1 The leader

To begin, the leader, once elected, acts as a message broker for the system. It receives all CRUD operations requests from all peers that need to be performed on the shared calendar. Requests are serialized when they arrive at the leader, ensuring a total ordering of operations. It is important to mention that each operation that changes the calendar state increments a logical clock. This logical clock is also stored in the Peer's checkpoint and log.

2.3.2 Followers

The follower has a simpler role than the leader; they listen for logical clock broadcasts from the leader and send their own CRUD operations to the leader for coordination. Each follower maintains its own logical clock. If a follower receives a logical clock broadcast from the leader and discovers its own clock is behind, it performs a sync operation to retrieve the leader's up-to-date calendar state and logical clock and atomically replaces its own state with the leader's state. The follower and leader logic are both implemented within the Server module as two distinct operating modes.

2.3.3 System Entrance

When a new peer joins the system, it encounters one of two scenarios:

- **The system has existing peers and a leader:** The new peer registers itself with the name server, queries the name server for existing peers, shuffles the resulting list of peer endpoints, and attempts a connection to each peer in the list. If any connection succeeds, the new peer sends a *who_is_leader* request, and the existing peer responds with the current leader's endpoint. Then, the new peer contacts the leader and performs a sync. This connection is saved on the new peer to be used for subsequent RPCs.

- **The system has no leader:** The new peer will attempt to contact other peers or the leader using the procedure previously described, but this will fail because there is no leader. In this case, the new peer will raise an election. The election proceeds using the Bully Algorithm, and the new peer will either coordinate with the new leader or win the election and become the leader.

2.3.4 Leader Election Scenarios

Several scenarios can trigger or result from a leader election, they are:

- **The leader dies:** if the leader fails to respond to a peer after 4 retries, that peer will trigger an election. Each peer attempts to find the peer with the highest PID. If a peer cannot find a peer with a higher PID, it assumes leadership. The newly elected leader sends a broadcast to receive the logical clock values from each peer in the system. If any peer has a higher logical clock than the current leader, the leader performs read operations to update its own calendar before beginning to handle requests.
- **An election is held while the leader is still alive:** If an election is initiated while the current leader is still running, the same Bully Algorithm logic applies. As soon as the old leader becomes aware of the election, it blocks all writes until it learns who the new leader is. The old leader participates in the election and may be re-elected if it has the highest PID. If a different peer is elected, the old leader's calendar state (which will have the highest logical clock) is transferred to the new leader.
- **A peer that just joined the system becomes the leader:** If the joining peer has the highest PID, it becomes the leader. It broadcasts a request for logical clocks, identifies the peer with the most up-to-date calendar, and synchronizes its state accordingly.

2.3.5 Fault Tolerance

The system allows peers to go offline and come back online. This produces two main fault tolerance scenarios:

- **Leader dies and comes back online with a higher logic clock:** When the old leader rejoins, it contacts the new leader, which has a lower logical clock but holds the agreed-upon authoritative calendar state. The old leader reads all events from the new leader's calendar, compares them with its own, and overwrites its local state to match the new leader's calendar. This design decision accepts that some data loss is possible (operations the old leader committed just before crashing may be lost if they were not propagated), but guarantees that all active peers agree on a single authoritative state.
- **Peer dies, comes back online:** The recovering follower finds the current leader, queries all calendar events, and updates its local state on any missing or changed events, then advances its logical clock based on the leader.

3 Correctness & Performance

3.1 Correctness of Goals

The main goals for this project were consistency, resilience & fault tolerance, and scalability & performance. For consistency, correctness was determined by CRUD operations correctly propa-

gating through the calendar system. The correctness test was performed manually by spinning up a number of peers and testing if CRUD operations would appear across all active peers in the system. For resilience and fault tolerance, the persistent calendar module was responsible for maintaining the calendar state in the event of a crash. Similarly, this was performed manually by killing and restarting peers, then ensuring all contents of the calendar were intact when the peer bootstrapped. Lastly, scalability became an issue when testing for performance. The project was limited to the maximum number of processes and threads that one could create on the student machines. Once reaching the limit, the performance testing was hindered as only 128 nodes were able to be spawned.

3.2 Server Throughput Performance

The throughput test was conducted on the leader peer and measured how many CRUD operations could be performed on the calendar state with a fixed number of events and a varying number of peers. The maximum number of operations sent to the leader peer was 256 create events. This constant factor was continually divided by the number of peers in the system, where each peer would send $256 / n$ create events to the leader node. As described above, the system reached user limits on the student machines, as there is a bottleneck on the number of threads that can spawn. Thus, the test caps out at 128 nodes in the system (129 including the peer) and each sends 2 events to the leader. The results of this strong scalability test are shown in Figure 3. We found these results surprising; we expected the throughput to increase as the server had higher utilization, then to level off once it was fully booked, then to drop off considerably once it was overloaded. However, our data show a decrease in throughput from 32 to 64 nodes, followed by an increase in throughput, with no visible drop off. We suspect this variability has to do with the non-deterministic behavior of nodes joining the system; since new peers contact random existing peers to determine the endpoint of the leader, it is possible to get unlucky and have a poor load distribution of *who_is_leader* requests, resulting in unpredictable bursts of network congestion. We noticed that in some test runs, the server logs would have many requests arrive concurrently, then relatively few requests, then many more, showing that server utilization was variable with the peer behavior. We also suspect that had we been able to scale our test harness to larger numbers of peers, we would have eventually seen the leader peer overload, and a throughput drop off.

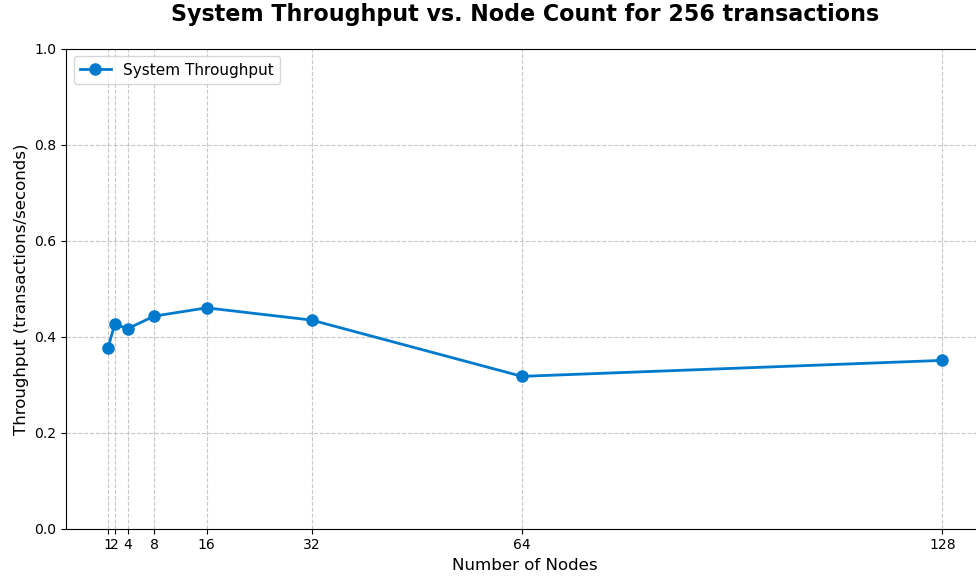


Figure 3: Networking working for Creating Calendar Event

3.3 Client Latency Performance

While conducting the throughput test for the leader peer, another test was running, tracking the average latency for each follower peer. This test has the same construction as the throughput test (e.g. 128 peers send 2 events, 64 peers send 4 events, ..., 1 peer sends 256 events). Again, the system was limited by the user limits on the student machines, hence why there is no testing done past 128 peers. The graph constructed from this test can be seen in Figure 4. These data suggest that a lower number of peers in the system has better latency than a larger number of peers. This makes sense because the system syncs on every operation performed on the leader peer's calendar. Thus, there is less synchronization being done for both the leader and the followers when there is a lower number of peers, leading to faster request turnaround from the leader peer. More specifically, for a system with 128 peers, the leader receives one create operation, broadcasts the logical clock, and then receives 128 syncs from each peer in the system. On the other hand, if there are 4 peers in the system, then the leader only receives Create request, it broadcasts the logical clock, then receives 4 syncs. Since the server is event-driven, it is able to respond more quickly to all the syncs and creates when there is a lower number of peers in the system, as there is a limit of 100 events pulled per select cycle.

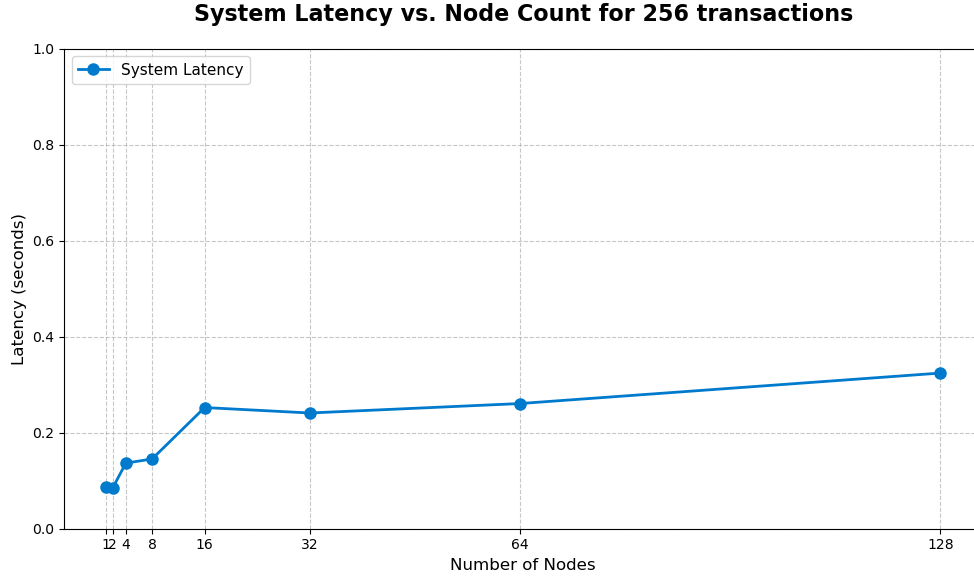


Figure 4: System Latency vs. Nodes

4 Limitations & Future work

The decentralized system built has three limitations described below:

4.1 Split Brain

The first limitation of the system is that it does not consider partitionability as one of its goals. The main focus for the project was consistency and availability, and based on the CAP theorem, the trade-off made was that the system can suffer from network partitions. In the event of a network partition, the peers separated from the leader will eventually send a CRUD operation over the network. This will result in a connection error that will be caught, and the peer will initiate an election. Once an election is conducted, a new leader will be elected, leading to two concurrent leaders for one shared calendar, and potentially two diverging calendar states. Once the network partition is resolved, conflicts don't arise until there is another election. In this scenario, one peer will be elected, and one of the calendar states will be propagated to all peers in the system. At this point, whichever calendar timeline has the highest logical clock will be kept, and the others will be discarded. For our implementation, there is no merge operation or way to consolidate differences between diverging calendar states when recovering from a network partition. Thus, the system will suffer data loss for a certain number of calendar events. We believe this problem could be mitigated by deploying a mechanism that prevents writes to the calendar during a partition, which would prevent diverging histories. This approach would hamper availability in favor of partitionability.

4.2 Election Live Lock

The second limitation of the system is the potential for a live lock. When testing the correctness of the system, we did not experience this problem, but based on the system architecture, we suspect that this issue might arise for a large network. In a scenario where the system has a large number of peers, and a peer cannot reach the leader, it will trigger an election and only reach out to peers with

higher PIDs. This peer will become dormant and only respond to messages telling it that another election has occurred or a leader has been found. Concurrently, another peer could join the system, or an existing peer could initiate a CRUD operation; the peer will reach out to the leader, fail, then start an election. At the same time, a leader could be elected, and it will send coordinated messages to everyone in the system. In this sequence, another election would be triggered, and all writes will be blocked in the system again. This situation could repeat itself several times over, resulting in the system operating but writes being constantly blocked. We believe this problem could be solved by introducing sleeps in the client in the event of multiple elections in short sequence; the sleeps would give the leader peer enough time to send *coordinate* messages to the whole network and break the election cycle.

4.3 UDP Broadcast Limitations

The third limitation is based on a design choice made early on in the architecture of the system. The design choice was made during the discussion of how to synchronize all peers in the system. We needed a mechanism for the leader to notify all peers of changes to the logical clock so that the peers could query the leader and sync to the authoritative calendar. There were two apparent options: spawn a daemon thread and establish a TCP connection to every peer in the system and send a sync message, or establish a UDP broadcast to signal to every peer in the system that a change has occurred. We implemented the latter option, which limited the scalability of the system. The use of UDP broadcast for logical clock synchronization restricts the system to a single network boundary, since IPV4 broadcasts will not propagate further than the local network. Depending on the network configuration and the host operating systems of the peers, this could even restrict the broadcast mechanism to a single subnet, significantly limiting its scalability and deployment flexibility. This limits the use of the calendar to peers located in the same LAN. We believe that a TCP-based replacement for the logical clock broadcast mechanism would solve this problem.

4.4 Future work

In future iterations of this project, implementing different leader election algorithms and testing the performance of the system would be an interesting experiment to see if different algorithms allow for better consistency and availability.